FLAC: Federated Learning with Autoencoder Compression and Convergence Guarantee

Mahdi Beitollahi and Ning Lu

Department of Electrical and Computer Engineering, Queen's University,
Kingston, ON, Canada, K7L 3N6
Email: beitollahi.mahdi@queensu.ca, ning.lu@queensu.ca

GLOBECOM 2022 - 2022 IEEE Global Communications Conference | 978-1-6654-3540-6/22/$31.00 ©2022 IEEE | DOI: 10.1109/GLOBECOM48099.2022.10000743

***Abstract*—*Federated Learning* (FL) is considered the key approach for privacy-preserving, distributed machine learning (ML) systems. However, due to the transmission of large ML models from users to the server in each iteration of FL, communication on resource-constrained networks is currently a fundamental bottleneck in FL, restricting the ML model complexity and user participation. One of the notable trends to reduce the communication cost of FL systems is gradient compression, in which techniques in the form of sparsification or quantization are utilized. However, these methods are pre-fixed and do not capture the redundant, correlated information across parameters of the ML models, user devices' data, and iterations of FL. Further, these methods do not fully take advantage of the error-correcting capability of the FL process. In this paper, we propose the Federated Learning with Autoencoder Compression (FLAC) approach that utilizes the redundant information and error-correcting capability of FL to compress user devices' models for uplink transmission. FLAC trains an autoencoder to encode and decode users' models at the server in the *Training State*, and then, sends the autoencoder to user devices for compressing local models for future iterations during the *Compression State*. To guarantee the convergence of the FL, FLAC dynamically controls the autoencoder error by switching between the Training State and Compression State to adjust its autoencoder and its compression rate based on the error tolerance of the FL system. We theoretically prove that FLAC converges for FL systems with strongly convex ML models and non-i.i.d. data distribution. Our extensive experimental results[1]over three datasets with different network architectures show that FLAC can achieve compression rates ranging from 83x to 875x while staying near 7 percent of the accuracy of the non-compressed FL systems.**

I. Introduction

Federated Learning (FL) was proposed in [1] to address the increasing privacy concerns of users without sacrificing the powerful capabilities of machine learning. In an FL system, a server trains a machine learning (ML) model by repeating the following process: i) each user device performs local training with its data using stochastic gradient descent (SGD) to minimize a global model. Then, they upload their local model updates to the FL server for aggregation; ii) the FL server aggregates the received local model and produces an improved global model; iii) and the server then sends the new updated global model to user devices, and user devices use this new global model as the reference. These steps create one global round, and global rounds are iterated until a desirable accuracy is reached.

FL can be deployed over communication networks to exploit the abundance of privacy-sensitive data and computational resources of user devices such as mobile devices and Internet of Things (IoT) devices. With these unique advantages, FL has been proposed for use in a variety of applications. For instance, FL for secure vehicular data sharing was proposed in [2], an FL-based localization scheme for mobile crowdsourcing was proposed in [3], and an FL-based scheme for mobile sensing in UAV networks was proposed in [4].

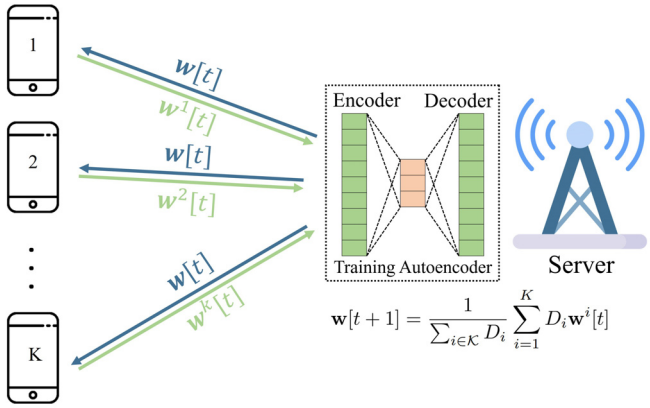
Even with all the promising benefits, high communication cost is one of the main concerns of real-world implementation of FL, especially in resource-constrained user devices. The reason is that in each global round of FL, user devices are required to upload their updated local model to the server and the size of updated models in each iteration can be extremely large, e.g., the VGGNet architecture has approximately 138 million parameters (4264 Mb) [5]. On the other hand, user devices have limited communication resources, e.g., the maximum LTE uplink rate is 75 Mb/s in ideal, laboratory settings [6]. It would take around a minute to transmit VGGNet in each iteration of FL using the LTE network in ideal conditions. This fundamental communication bottleneck in FL is restricting the ML model complexity and user participation.

To reduce the communication cost, researchers have utilized classic compression techniques such as Sparsification and Quantization. However, these techniques are pre-fixed and non-adaptive and do not take advantage of the unique characteristics of FL, such as its iterative error-correcting nature or FL's redundant information across its user devices, model parameters, or FL iterations. The error-correcting characteristic of FL comes from the fact that the SGD algorithm used by users is an inherently noisy process, and SGD can compensate for some errors in each iteration. FL's redundant information characteristic is due to the correlation of the data across users, which also causes the correlation between model parameters and iterations of FL.

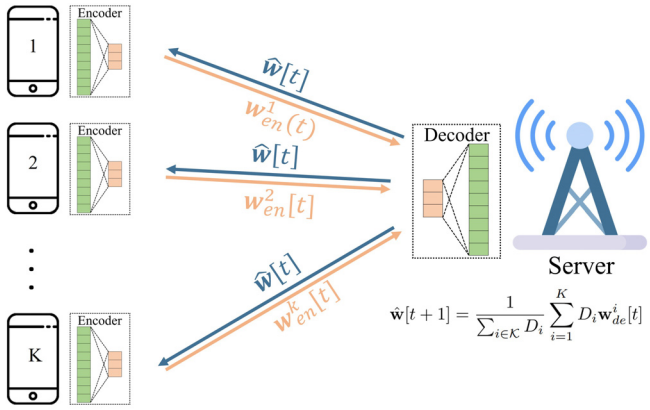
To utilize the redundant information across an FL system and compress the size of the updated models dynamically for uplink transmission, we propose a novel **Federated Learning with Autoencoder Compression** (FLAC) approach that utilizes an autoencoder to compress and encode updated local

[1]All code and experiments are publicly available at: https://github.com/mahdibeit/FLAC.

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:08:16 UTC from IEEE Xplore. Restrictions apply.



(a) Training State



(b) Compression State

Fig. 1: Overview of the FLAC approach. FLAC dynamically switches between the Training State (a) and Compression State (b) based on the error tolerance of the FL.

models. FLAC trains the autoencoder at the server using the updated local models of the users as shown in Figure 1. Then, FLAC sends the autoencoder to users to utilize its encoder part and compress the local models for the next iterations, and the server utilizes the decoder part of the autoencoder to decode compressed users' updated local models and calculate the global model. To guarantee the convergence of the global model, FLAC dynamically switches between the *Training State* and *Compression State* to adjust its error based on the error tolerance of the FL, which is determined by the learning rate. FLAC reduces an FL system's uplink transmission costs at the expense of additional computation as well as additional downlink transmission costs. Reducing the size of the uplink at the cost of the downlink is favorable for resource-constrained FL systems since their uplink transmission resources are more constrained in terms of both availability and power cost compared to the downlink [7]. The main contributions of the paper include:

- We propose FLAC as a novel compression approach that utilizes the redundant, correlated information across FL's user devices, model parameters, and iterations of an FL system to compress the size of the updated local models for uplink transmission.
- We enable FLAC to dynamically switch between the *Training State* and *Compression State* based on the error tolerance of the FL system, and we theoretically prove that FLAC converges for FL systems with strongly convex ML models and non-i.i.d. data distribution.
- Finally, we provide experimental results over three datasets with different network architectures to show the effectiveness of FLAC.

The rest of this paper is organized as follows. In Section II, we discuss related works. The system model and problem formulation are described in Section III. FLAC is introduced in Section IV. In Section V, we provide our experimental results, and Section VI concludes the paper.

II. Related Works

A wide variety of methods have been proposed to reduce the communication cost of an FL network. We categorize and summarize the substantial research on communication-efficient FL into three different groups.

1) **Reducing the frequency of transmissions**: These methods reduce the amount of communication between the server and users. For instance, in FedAVG [1], each users performs multiple rounds of local SGD instead of only one to compute the local model before sending the local model to the server. At the cost of computation resources, it was observed that this method does not affect the convergence rate of the FL model significantly. However, in practice, it is hard to choose the appropriate number of local rounds as it may vary a lot for different models.
2) **Sparsification**: These methods reduce the entropy of the updates by restricting changes to only a subset of the parameters. In this regard, a random projection matrix is utilized in [8] to reduce the dimension of the weights and only send the highest weights to sever. In this lossy compression method, the rest of the weights are accumulated in a residual. Sparsification fails to capture and utilize the redundant information across the FL system.
3) **Quantization**: Quantization methods reduce the entropy of the weight updates by restricting all updates to a reduced set of values. For instance, SignSGD was proposed in [9] to quantize every gradient update to its binary sign, therefore, reducing the bit size per update. To extend the pre-fixed notion of signSGD and minimize the unused quantized space, only the region occupied by the weights using their maximum and minimum weights was dynamically quantized in [10]. Even though these methods have theoretical convergence guarantees for i.i.d. data distribution, their performance does not match the sparsification methods.
4) **Model Compression**: This approach changes the user's model to reduce the number of weights required to be transmitted for FL. In this regard, Knowledge Distillation is proposed in [11] in which the original model is transferred to a smaller model, and therefore, this smaller model could be utilized for FL at the cost of losing the encapsulated information in the main model. As another

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:08:16 UTC from IEEE Xplore. Restrictions apply.

approach, Federated Dropout was proposed in [12] in which the original model is replaced with sub-models by zeroing out a fixed number of activation nodes at each fully connected layer. These model compression methods usually cannot be generalized since rebuilding a smaller model vary for different model architectures.

To take advantage of the benefits of the described methods and balance their trade-offs, we propose FLAC as a dynamic autoencoder-based compression method that captures and utilizes the redundant information to reduce the communication cost and guarantee its convergence. There is a limited investigation into compressing another model using an autoencoder. The closest works to ours is [13] and [14]. However, the autoencoder utilized in [14] is trained over artificial random data and does not utilize the data or model-specific information. The autoencoder in [13] is trained on each user, and therefore, does not capture the redundant information across users and their models. Furthermore, no guarantee of convergence has been provided by these works.

III. System Model

FL's goal is to learn an ML model with reshaped parameters in a vector $\mathbf{w} \in \mathbb{R}^d$ from data stored across K users in the set $\mathcal{K}$, where d represents the number of parameters in the ML model. We represent scalar D_i as the size of the local data for user $i \in \mathcal{K}$. The server distributes the current global model weights $\mathbf{w}[t]$ to K users' devices in each global round $t \geq 0$. These user devices independently update their model by performing local computations such as multiple steps of SGD based on their local data to minimize their local model F_i. Let the updated local models be $\mathbf{w}^1[t], \mathbf{w}^2[t], ..., \mathbf{w}^K[t]$. Then, each selected user uploads the update back to the server, where the global update is computed by aggregating all the user's local updates as follows:

$$\mathbf{w}[t+1] = \frac{1}{\sum_{i\in\mathcal{K}} D_i} \sum_{i=1}^{K} D_i \mathbf{w}^i[t]. \tag{1}$$

Then, the server sends back $\mathbf{w}[t+1]$ as the new global model to the user devices to complete one global round of FL. This process is iterated until the ML model reaches a pre-defined accuracy. The learning objective is to find the best model parameter $\mathbf{w}^*$ that minimizes the global loss function, i.e., $\mathbf{w}^* = \arg\min_{\mathbf{w}} F(w)$, where $F(\mathbf{w}) = \frac{1}{\sum_{i\in\mathcal{K}} D_i} \sum_{i\in\mathcal{K}} D_i F_i(\mathbf{w})$. Let F^* and F_i^* represent the minimum value of F and F_i, respectively. Then, $\Gamma = F^* - \frac{1}{\sum_{i\in\mathcal{K}} D_i} \sum_{i\in\mathcal{K}} D_i F_i^*(\mathbf{w})$ quantifies the degree of non-i.i.d. distribution of the data.

Our goal is to reduce the communication cost of uploading $\mathbf{w}^i[t]$ for each user $i \in \mathcal{K}$ by compression and utilizing the redundant information across devices and parameters of the users' models.

IV. FL with autoencoder compression (FLAC)

To reduce the communication cost of uploading $\mathbf{w}^i[t]$ by compression, we use a single autoencoder to encode and learn the features in the users' local models. This autoencoder is comprised of an encoder and a decoder, and it is trained to recreate the user's local model (input) via an intermediate compressed state as shown in Figure 1a. The encoding network can be represented by a standard neural network function f_{en} as follows:

$$\mathbf{w}_{en}^i[t] = f_{en}(\mathbf{w}^i[t]) \quad i \in \mathcal{K}, \tag{2}$$

where $\mathbf{w}_{en}^i$ is the latent dimension and compressed version of $\mathbf{w}^i[t]$. User devices utilize the encoder part of this autoencoder, f_{en} to compress their local model and send the compressed local model, $\mathbf{w}_{en}^i$, to the server as shown in Figure 1b. Similarly, the decoder network can be represented as follows:

$$\mathbf{w}_{de}^i[t] = f_{de}(\mathbf{w}_{en}^i[t]) \quad i \in \mathcal{K}. \tag{3}$$

The server uses the decoder part of the autoencoder, f_{de}, to approximate the original values of user i's local model based on the transmitted latent vector $\mathbf{w}_{en}^i$ and calculates the new global model as follows:

$$\hat{\mathbf{w}}[t+1] = \frac{1}{\sum_{i\in\mathcal{K}} D_i} \sum_{i=1}^{K} D_i \mathbf{w}_{de}^i[t]. \tag{4}$$

To approximate $\mathbf{w}_{de}^i[t]$ close to $\mathbf{w}^i[t]$, we train the autoencoder using the $\mathcal{L}_2$-norm loss function for $\forall i \in \mathcal{K}$ as follows:

$$\mathcal{L}_2(\mathbf{w}^i, \mathbf{w}_{de}^i) = \left\|\mathbf{w}^i - \mathbf{w}_{de}^i\right\|^2 \tag{5a}$$

$$= \left\|\mathbf{w}^i - f_{de}(f_{en}(\mathbf{w}^i)\right\|^2. \tag{5b}$$

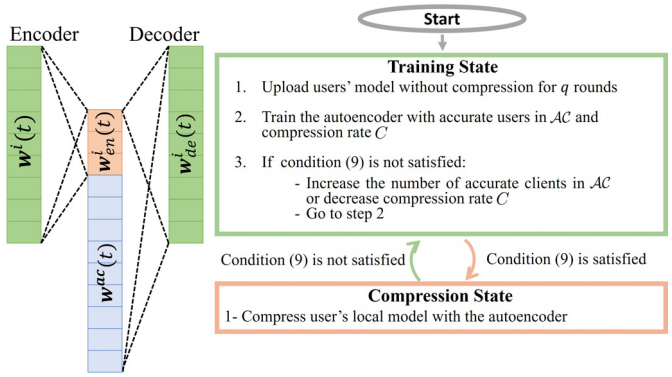
In the training state, to train the autoencoder at the server, we let the FL system do its process as usual by transmitting the actual values of the local weights without compressing them, and we use those values to train, validate, and test the autoencoder using (5), as shown in Figure 1a. Then, by entering the compression state, we send the autoencoder to the user devices, instructing them to use the encoder, f_{en}, to compress the local models before transmission, as illustrated in Figure 1b.

We employ a fully connected autoencoder with one hidden layer and a ReLU activation function, the size of which dynamically adapts based on the global model size, i.e., $|\mathbf{w}|$. We refer to the compression rate as C, which is defined as $C = \frac{|\mathbf{w}|}{|\mathbf{w}_{en}|}$. The size of the hidden layer determines the compression rate and can be adjusted dynamically based on the compression objectives. Increasing the compression rate increases the loss of information and reduces the model's capability to recreate the inputs accurately.

A. Autoencoder with accurate users

To further increase the autoencoder accuracy, we propose a new method and consider a case where a subset of users, $\mathcal{AC} \subseteq \mathcal{K}$ are connected to the server and send their local models without compression through the FL process. The server can utilize these accurate local models during the decoding process to reconstruct local models of other users, as shown in Figure

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:08:16 UTC from IEEE Xplore. Restrictions apply.



(a) Autoencoder with accurate users (b) Illustration of FLAC

Fig. 2: The autoencoder with accurate users (a), and illustration of FLAC (b).

2a. In this scenario, the decoding network can be represented as follows:

$$\mathbf{w}^i_{de}[t] = f_{de}(\mathbf{w}^i_{en}[t], \mathbf{w}^{ac}[t]) \quad i \in \mathcal{K} \setminus \mathcal{AC}, \tag{6}$$

where $\mathbf{w}^{ac}[t]$ is a vector containing local models of accurate users, i.e., $\mathbf{w}^{ac}[t] = (\mathbf{w}^{ac_1}[t], ..., \mathbf{w}^{ac_N}[t])$ where $ac_1, ..., ac_N \in \mathcal{AC}$. Utilizing this accurate information, the server can then calculate the global model as follows:

$$\hat{\mathbf{w}}[t+1] = \frac{1}{\sum_{i\in\mathcal{K}} D_i}\Big(\sum_{i\in\mathcal{K}\setminus\mathcal{AC}} D_i \mathbf{w}^i_{de}[t] + \sum_{i\in\mathcal{AC}} D_i \mathbf{w}^i[t]\Big). \tag{7}$$

For the architecture of the autoencoder with accurate information, we concatenate accurate models to the compressed models of other users, i.e., $\mathbf{w}^i_{ac-en}[t] = (\mathbf{w}^i_{en}[t], \mathbf{w}^{ac}[t])$ in the hidden layer, as depicted in Figure 2a. Intuitively, accurate users enable the autoencoder to better track and keep up during the FL process.

B. FLAC

The autoencoder compression method described in this section is inherently a lossy-compression method and its errors are dependent on the compression rate, the number of accurate users, and the training process (solving the optimization problem (5)). However, FL can tolerate some errors due to its error-correcting iterative nature as it re-minimizes the global loss function in each global round. Therefore, the question is how much FL can tolerate errors and how can we adjust the compression error based on this tolerance to guarantee the convergence of the FL process with autoencoder compression.

It was proved in [15] that in FL systems with strongly convex global models, FedAVG can maintain the same convergence rate of $\mathcal{O}(1/T)$ as long as the error's variance in uplink is decreasing with the rate of $\mathcal{O}(1/t^2)$ and it is zero-mean. Basically, the idea is that if the error is less than the learning rate of the FL users, users can compensate for those errors in the next global rounds of FL. Therefore, we utilize the same idea and introduce FLAC which is an approach that controls the autoencoder compression error using the accurate users and compression rate throughout the FL process to make sure that the error is less than the learning rate of the users and the FL can compensate for this error.

To calculate the autoencoder error throughout the FL process, we consider the instantaneous errors and define the error in user i as follows:

$$\mathbf{e}^i[t] = \mathbf{w}^i[t] - \mathbf{w}^i_{de}[t]. \tag{8}$$

To guarantee the convergence of FL with the autoencoder statistically, we need to make sure that the autoencoder satisfies the following conditions.

$$i. \quad \mathbb{E}\left[\left\|\mathbf{e}^i\right\|^2\right] \le e^{th}[t] \le \eta_t^2 \sim \mathcal{O}\left(\frac{1}{t^2}\right) \quad \forall i \in \mathcal{K}, \tag{9a}$$

$$ii. \quad \mathbb{E}\left[\mathbf{e}^i\right] = \mathbf{0} \quad \forall i \in \mathcal{K}, \tag{9b}$$

where η is the learning rate and $e^{th}[t] \sim \mathcal{O}\left(\frac{1}{t^2}\right)$ is a pre-fixed function. To approximate these statistical values using their instantaneous values, we use windows of size q, i.e., $\mathbb{E}\left[\mathbf{e}^i\right] \simeq \frac{1}{q}\sum_{l=t-q}^{t} \mathbf{e}^i[l]$ and $\mathbb{E}\left[\left\|\mathbf{e}^i\right\|^2\right] \simeq \frac{1}{q}\sum_{l=t-q}^{t} \mathbb{E}\left[\left\|\mathbf{e}^i[l]\right\|^2\right]$.

Algorithm 1 FLAC

Parameters: size of the window q, current iteration t, compression rate C, accurate users $\mathcal{AC}$ = {}, and e^{th}
Initialization: State ← Training State
if State is Training State **then**
  for q iterations **do**
    $\mathbf{w}[t+1] = \frac{1}{\sum_{i\in\mathcal{K}} D_i}\sum_{i=1}^{K} D_i \mathbf{w}^i[t]$
  end for
  while conditions in (9) is not satisfied **do**
    Train the autoencoder using (5)
    Add users to $\mathcal{AC}$ or decrease C
  end while
  Send the autoencoder to users
  State ← Compression State
end if
if State is Compression State **then**
  for q iterations **do**
    Calculate the global model using compressed local models using (7)
  end for
  while conditions in (9) satisfies, for each iteration **do**
    Encode user's models with the autoencoder using (2) and send it to the server
  end while
  State ← Training State
end if

The detailed FLAC process is described in Algorithm 1 and illustrated in Figure 2b. In FLAC, the FL system starts its process in the *Training State* in which without any compression, it does the FL process for q rounds of global iterations. Then, it utilizes those q rounds to train, validate, and test the autoencoder described in this section. In the server, if the error of the autoencoder satisfies the condition in (9), we send the autoencoder to the user devices to compress their local model for the next iterations in the *Compression State*. Otherwise, we

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:08:16 UTC from IEEE Xplore. Restrictions apply.

reduce the compression rate or add accurate users if possible until we satisfy those conditions. These conditions make sure that the trained autoencoder's error is less than the learning rate. In the Compression State, after first q iterations, in each iteration, we check the conditions in (9) using the previous q iterations. If the conditions are not satisfied, we switch to the Training State and re-train the autoencoder from scratch. These conditions assure that the FL still can compensate for the autoencoder's error.

FLAC dynamically switches between the Training State and Compression State and adapts itself based on the different stages of the FL process. For instance, during the first iterations of FL, FL's learning rate is high and users are taking large steps towards the optimum point. In this first stage, small errors in the user's models do not affect the performance since the model is going to take another big step towards the optimum point in the next iterations. However, when FL's learning rate is small, even small errors can cause a gap. FLAC dynamically adjusts its error based on the error tolerance of the FL to compress the models as much as possible. Using accurate users guarantees that there exists an autoencoder with zero error that satisfies the conditions in 9 for all learning rates no matter how small. The additional processing cost of FLAC on the users that utilize compression is $|\mathbf{w}| * |\mathbf{w}_{en}| + (|\mathbf{w}_{en}| + |\mathbf{w}_{ac}|) * |\mathbf{w}|$ FLOPs.

C. Convergence Guarantee

The main theoretical result is that FLAC converges to the global optimum at a rate of $\mathcal{O}(\frac{1}{T})$, which is the same scaling behavior of FedAVG [1].

To simplify the analysis, we assume that the local dataset sizes of of all user are same, i.e., $D_i = D_j$ for $\forall i, j \in \mathcal{K}$. We assume in each iteration we are selecting N users for FL and each user runs the SGD for E epochs. Further, we make the following assumption that are widely used for the convergence analysis (see [15]) as follows:

1) **L-smooth:** $\forall \mathbf{v}, \mathbf{w}, F(\mathbf{v}) \leq F(\mathbf{w}) + (\mathbf{v} - \mathbf{w})^T \nabla F(\mathbf{w}) + \frac{L}{2} \|\mathbf{v} - \mathbf{w}\|^2$;
2) **μ-strongly convex:** $\forall \mathbf{v}, \mathbf{w}, F(\mathbf{v}) \geq F(\mathbf{w}) + (\mathbf{v} - \mathbf{w})^T \nabla F(\mathbf{w}) + \frac{\mu}{2} \|\mathbf{v} - \mathbf{w}\|^2$;
3) **Bounded gradient and bounded variance of gradient:** $\mathbb{E}[\|\nabla F(\mathbf{w}^k[t], \xi^k[t])] - \nabla F(\mathbf{w}^k[t])\|^2] \leq \sigma^2$ and $\mathbb{E}[\|\nabla F(\mathbf{w}^k[t], \xi^k[t])]\| \leq G^2$.

Theorem IV.1. *Consider the system model in Section III and IV. When the above assumptions hold, choose $\kappa = \frac{L}{\mu}$, $\gamma = \max\{8\kappa, E\}$ and the learning rate $\eta = \frac{2}{\mu(\gamma+t)}$. If the autoencoder error threshold in FLAC scales such that:*

$$e^{th}[t] \leq \eta_t^2 = \frac{4}{\mu^2(\gamma + t - 1)^2} \sim \mathcal{O}\left(\frac{1}{t^2}\right), \tag{10a}$$

$$\mathbb{E}\left[\mathbf{e}^i\right] = \mathbf{0} \quad \forall i \in \mathcal{K}. \tag{10b}$$

Then, FLAC with non-i.i.d. data distribution and full device participation satisfies

$$\mathbb{E}\left[F(\hat{\mathbf{w}}[T])\right] - F(\mathbf{w}^*)] \leq \frac{\kappa}{\gamma + T - 1}\left(\frac{2B}{\mu} + \frac{\mu\gamma}{2}\left[\|\mathbf{w}[1] - \mathbf{w}^*\|^2\right]\right), \tag{11}$$

where
$B = \sum_{i=1}^{N} \frac{\sigma_i^2}{N^2} + 6L\Gamma + 8(E-1)G^2 + \mathbb{E}[\|\mathbf{e}^{th}\|^2]$.

Proof. Using the smoothness of F, we can write the gap as follows:

$$\mathbb{E}[F(\hat{\mathbf{w}}[t])] - F(\mathbf{w}^*) \leq \frac{L}{2}\,\mathbb{E}\left[\|\hat{\mathbf{w}}[t] - \mathbf{w}^*\|^2\right]. \tag{12}$$

We will exploit the results in [15] in which they evaluate the convergence rate of FL with both downlink and uplink zero-mean and independent error. However, the uplink errors from different users are not independent in FLAC. To address this, we bound the uplink error term as follows:

$$\mathbb{E}\left[\|\mathbf{w}[t] - \hat{\mathbf{w}}[t]\|^2\right] = \mathbb{E}\left[\|\mathbf{e}[t]\|^2\right] = \frac{1}{N^2}\,\mathbb{E}\left[\left\|\sum_{i\in\mathcal{N}} \mathbf{e}^i[t]\right\|^2\right]$$
$$\leq \frac{1}{N^2}\left[N^2 \left\|\max_{i\in N} \mathbb{E}\left[\mathbf{e}^i[t]\right]\right\|^2\right] \leq e^{th}[t].$$

Using the results in [15], the gap is given by

$$\mathbb{E}\left[\|\hat{\mathbf{w}}[t+1] - \mathbf{w}^*\|^2\right] \leq (1 - \eta_t\mu)\,\mathbb{E}\left[\|\hat{\mathbf{w}}[t] - \mathbf{w}^*\|^2\right]$$
$$+ e^{th}[t] + \eta_t^2\left[\sum_{i=1}^{N} \frac{\sigma_i^2}{N^2} + 6L\Gamma + 8(E-1)G^2\right].$$

Denote $\Delta_t = \mathbb{E}[\|\hat{\mathbf{w}}[t+1] - \mathbf{w}^*\|^2]$. If we set the FLAC's error such that $e^{th}[t] \leq \eta_t^2$, we always have $\Delta_{t+1} \leq (1 - \eta_t\mu)\Delta_t + \eta_t^2 B$, where $B = \sum_{i=1}^{N} \frac{\sigma_i^2}{N^2} + 6L\Gamma + 8(E-1)G^2 + \mathbb{E}[\|\mathbf{e}^{th}\|^2]$. We set the learning rate $\eta_t = \frac{\beta}{t+\gamma}$ for some $\beta \geq \frac{1}{\mu}$ and $\gamma \geq 0$ such that $\eta_1 \leq \min\{\frac{1}{\mu}, \frac{1}{4L}\} = \frac{1}{4L}$ and $\eta_t \leq 2\eta_{t+E}$. By induction, it is easy to verify for $t = 1$ and prove that $\Delta_t \leq \frac{v}{\gamma+t}$, where $v = \max\{\frac{\beta^2 B}{\beta\mu - 1}, (\gamma+1)\Delta_0\}$ as follows:

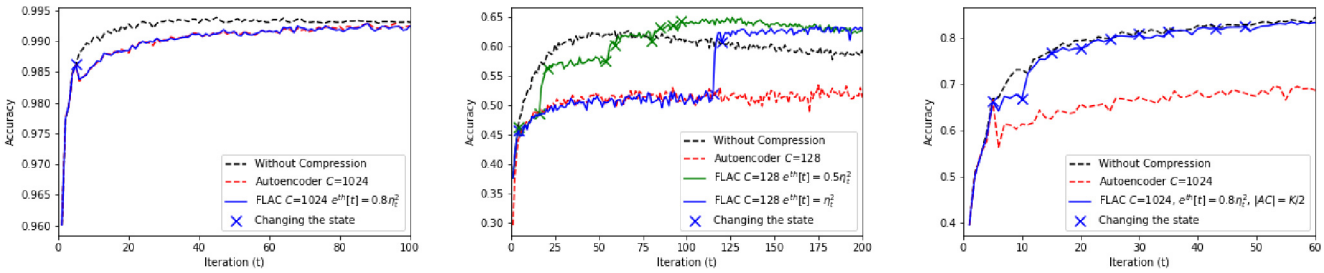
$$\Delta_{t+1} \leq (1 - \eta\mu)\Delta_t + \eta_t^2 B = \left(1 - \frac{\beta\mu}{t+\gamma}\right)\frac{v}{t+\gamma} + \frac{\beta^2 B}{(t+\gamma)^2}$$
$$\leq \frac{v}{t+\gamma+1}.$$

Then by using Δ_t in the equation (12) and setting $t = T$, it is easy to verify the claim. ■

V. Results and Analyses

For the experiments, we use convolutional neural networks (CNNs) trained on MNIST [16], CIFAR-10 [17], and Speech commands dataset [18], and we use the same CNN architectures as [19] (582026 parameters), [1] (62006 parameters), and [20] (27303 parameters) for each dataset respectively to evaluate different model architectures. For implementation, we utilize Flower [19] framework and 10 users with i.i.d. data distribution. For training the autoencoder, we use Adam and 100 epochs of iterations. We set $q = 5$, the number of epochs at $E = 1$, and the learning rate at $\eta = 0.1$ with decay rate $= 0.9995$. We set the approximation error for the mean condition in (9) at 0.1. For simplicity, we only compress the largest layers that contributed the most to the size of each model from each architecture, i.e., 77 percent for CIFAR-10, 90 percent for MNIST, and 45 percent for the Speech dataset.

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:08:16 UTC from IEEE Xplore. Restrictions apply.



(a) MNIST (b) CIFAR-10 (c) Speech

Fig. 3: Accuracy performance of the FLAC over MNIST (a), CIFAR-10 (b) and Speech dataset (c).

Figure 3 shows the performance of FLAC over three datasets. This figure shows that when we use a fixed autoencoder trained over only the first q iterations throughout the FL process the autoencoder reduces the slope of accuracy. The reason is that its error becomes larger than the change of the model in the next iterations. Basically, it would neglect all the small changes of previous iterations. Figure 3 shows that FLAC addresses this issue by retraining the autoencoder whenever its error is above e^{th}.

Figure 3 also shows FLAC's performance with different e^{th}s. Increasing the eth allows FL to tolerate more errors without retraining the autoencoder and increase its accuracy gap with an FL without compression. On the other hand, small e^{th} reduces the FLAC's accuracy gap with an FL without compression at the cost of more computation since FLAC needs to retrain the autoencoder even for small errors.

For CIFAR-10, we manage to compress 77 percent of parameters in 85 percent of iterations with the rate of $C = 128$ while staying near 7 percent of the non-compressed accuracy throughout the FL process as shown in Figure 3b. This results in an 83x compression rate while staying near 7 percent of the non-compressed accuracy throughout the FL process. Similarly, as shown in Figure 3a and Figure 3c, we achieve compression rate of 875x and 133x times while staying near 3 and 7 percent of the non-compressed accuracy for MNIST and Speech dataset respectively.

VI. Conclusion

In this paper, we proposed FLAC as a novel compression method with convergence guarantee that utilizes the redundant information across the FL system to compress the size of the updated local models dynamically for uplink transmission. FLAC dynamically adjusts its autoencoder based on the error tolerance of the FL. Finally, we presented our experimental results to show the effectiveness of FLAC.

References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. PMLR, 2017, pp. 1273–1282.

[2] Y. Lu, X. Huang, Y. Dai, S. Maharjan, and Y. Zhang, "Differentially private asynchronous federated learning for mobile edge computing in urban informatics," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 3, pp. 2134–2143, 2019.

[3] W. Li, C. Zhang, and Y. Tanaka, "Pseudo label-driven federated learning-based decentralized indoor localization via mobile crowdsourcing," *IEEE Sensors Journal*, vol. 20, no. 19, pp. 11 556–11 565, 2020.

[4] Y. Wang, Z. Su, N. Zhang, and A. Benslimane, "Learning in the air: Secure federated learning for uav-assisted crowdsensing," *IEEE Transactions on network science and engineering*, vol. 8, no. 2, pp. 1055–1069, 2020.

[5] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2736–2744.

[6] Z. Shen, A. Papasakellariou, J. Montojo, D. Gerstenberger, and F. Xu, "Overview of 3gpp lte-advanced carrier aggregation for 4g wireless communications," *IEEE Communications Magazine*, vol. 50, no. 2, pp. 122–130, 2012.

[7] H. Elshaer, F. Boccardi, M. Dohler, and R. Irmer, "Downlink and uplink decoupling: A disruptive architectural design for 5g networks," in *2014 IEEE global communications conference*. IEEE, 2014, pp. 1798–1803.

[8] M. M. Amiri and D. Gündüz, "Federated learning over wireless fading channels," *IEEE Transactions on Wireless Communications*, vol. 19, no. 5, pp. 3546–3557, 2020.

[9] J. Bernstein, Y.-X. Wang, K. Azizzadenesheli, and A. Anandkumar, "signsgd: Compressed optimisation for non-convex problems," in *International Conference on Machine Learning*. PMLR, 2018, pp. 560–569.

[10] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "Qsgd: Communication-efficient sgd via gradient quantization and encoding," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[11] G. Hinton, O. Vinyals, J. Dean *et al.*, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, vol. 2, no. 7, 2015.

[12] S. Caldas, J. Konečny, H. B. McMahan, and A. Talwalkar, "Expanding the reach of federated learning by reducing client resource requirements," *arXiv preprint arXiv:1812.07210*, 2018.

[13] S. Chandar, P. Chandran, R. Bhat, and A. Chakravarthi, "Communication optimization in large scale federated learning using autoencoder compressed weight updates," *arXiv preprint arXiv:2108.05670*, 2021.

[14] H. Li and T. Han, "An end-to-end encrypted neural network for gradient updates transmission in federated learning," *arXiv preprint arXiv:1908.08340*, 2019.

[15] X. Wei and C. Shen, "Federated learning over noisy channels: Convergence analysis and design examples," *IEEE Transactions on Cognitive Communications and Networking*, 2022.

[16] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[17] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[18] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," *arXiv preprint arXiv:1804.03209*, 2018.

[19] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, T. Parcollet, P. P. de Gusmão, and N. D. Lane, "Flower: A friendly federated learning research framework," *arXiv preprint arXiv:2007.14390*, 2020.

[20] W. Dai, C. Dai, S. Qu, J. Li, and S. Das, "Very deep convolutional neural networks for raw waveforms," in *2017 IEEE international conference on acoustics, speech and signal processing (ICASSP)*. IEEE, 2017, pp. 421–425.

Authorized licensed use limited to: Queen's University. Downloaded on March 12,2026 at 00:08:16 UTC from IEEE Xplore. Restrictions apply.